

Transformer: Attention으로 sequence를 연결하다

Attention Is All You Need

Ashish Vaswani et al. (2017)

Primary technical text: [arXiv:1706.03762v7](#) (2023-08-02); original proposal: 2017

먼저 판본을 고정한다

같은 제목 아래 arXiv revision과 proceedings artifact가 공존한다

Artifact	Date	이 강의의 역할
arXiv v1	2017-06-12	별도 version supplement
NeurIPS proceedings	2017	독립 publication record
arXiv v7	2023-08-02	Canonical mechanism source

수치와 공식은 paper title만이 아니라 artifact와 위치까지 함께 인용한다

문제는 token 계산의 의존 순서였다

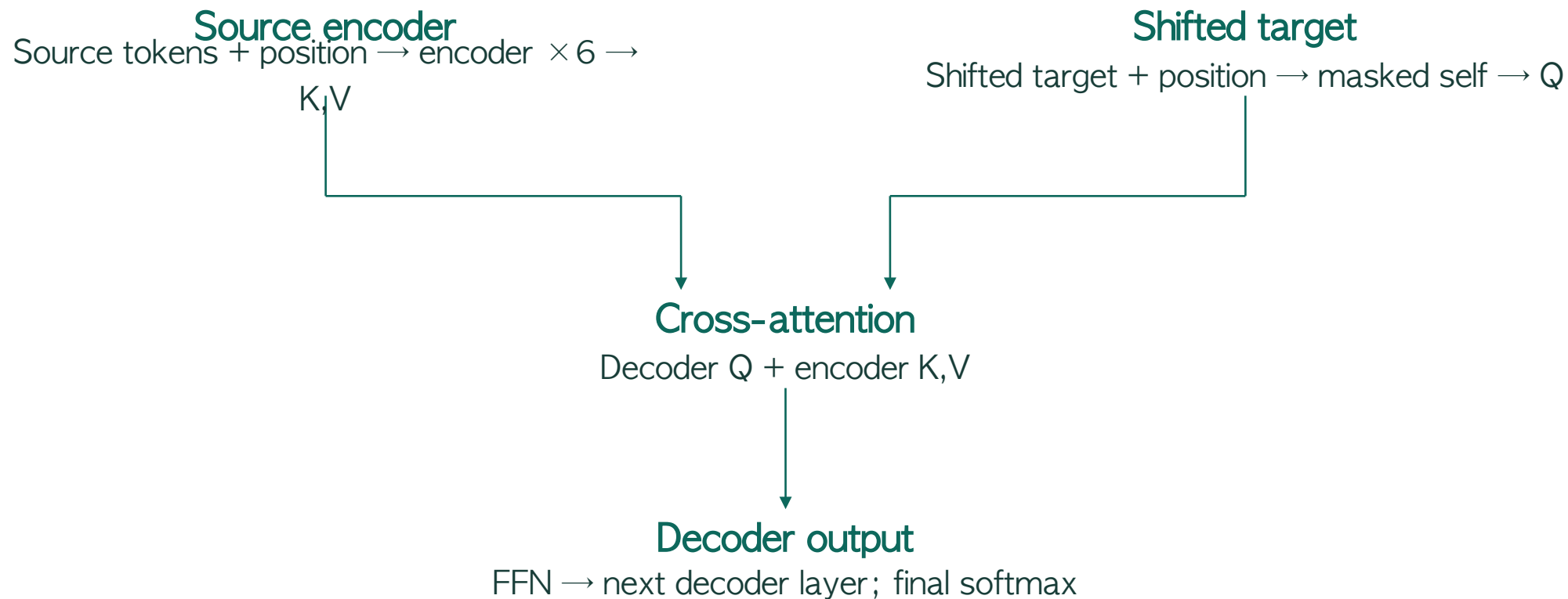
RNN은 이전 state를 기다리지만 self-attention layer는 모든 위치의 표현을 함께 계산할 수 있다

Layer	Sequential ops	Max path	Per-layer cost
Recurrent	$O(n)$	$O(n)$	$O(n d^2)$
Self-attention	$O(1)$	$O(1)$	$O(n^2 d)$
Restricted attention	$O(1)$	$O(n/r)$	$O(r n d)$

짧은 sequential depth와 작은 total cost는 같은 주장이 아니다

Source와 target이 decoder layer에서 만난다

교육용 예시 source는 The cat sleeps, target prefix는 Le chat이다



Encoder layer 하나를 펼쳐 보기

동일한 구조를 N=6회 쌓으며 각 sublayer에 residual connection과 layer normalization을 적용한다

01	Layer input	모든 source token representations
↓		
02	Self-attention	Q, K, V가 같은 encoder input에서 옴
↓		
03	Add & Norm	$\text{LayerNorm}(x + \text{Sublayer}(x))$
↓		
04	Position-wise FFN	각 position에 같은 network 적용
↓		
05	Add & Norm	Residual을 더해 다음 layer로 전달

Attention-only는 FFN·residual·normalization이 없다는 뜻이 아니다

Scaled dot-product attention의 입력

Q와 K는 compatibility를 계산하고 V는 그 weight로 섞을 정보를 담는다

Symbol	Shape role	계산에서 하는 일
Q	Query rows	각 위치가 필요한 정보 질의
K	Key rows	Query와 dot product 비교
V	Value rows	Softmax weight로 합산
d_k	Key dimension	Score scaling의 기준

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

숫자로 재현하는 attention

교육용 예시이며 논문의 reported experiment가 아니다: $d_k=2$, $q=[1,0]$

Step	Input	Result	Meaning
Dot	$k_1=[1,0]$, $k_2=[0,1]$	$[1, 0]$	Compatibility
Scale	divide by $\sqrt{2}$	$[0.7071, 0]$	Large score 완화
Softmax	exp 후 normalize	$[0.6698, 0.3302]$	Weight 합은 1
Weighted sum	$v_1=[10,0]$, $v_2=[0,10]$	$[6.698, 3.302]$	두 value의 혼합

Attention output은 가장 큰 value 하나를 고르는 hard selection이 아니다

왜 $\sqrt{d_k}$ 로 나누는가

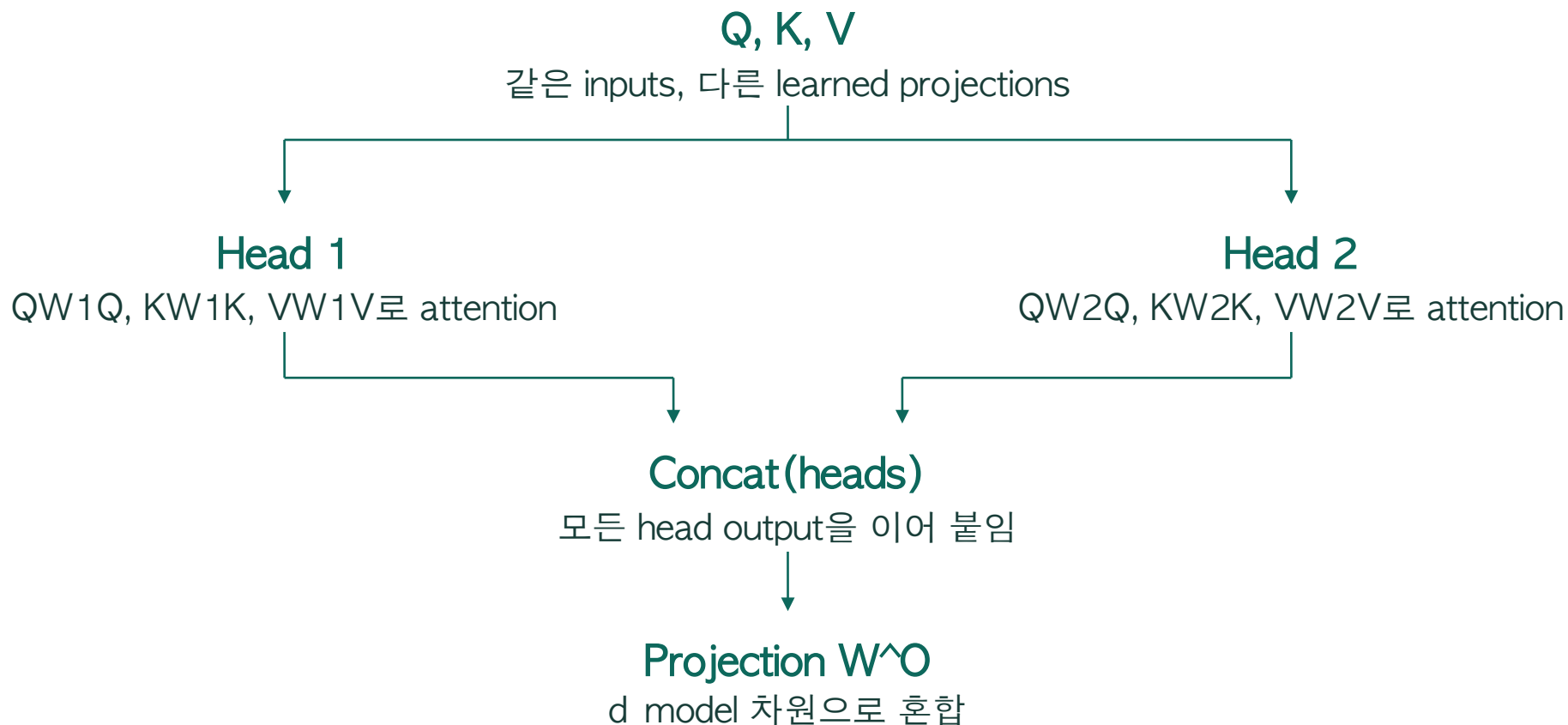
독립적인 query/key 성분의 평균이 0, 분산이 1이라는 설명 조건을 따른다

01	Dot product	d_k 개 component product를 더한다
↓		
02	Variance grows	가정 아래 score variance가 d_k 가 된다
↓		
03	Large magnitude	d_k 가 크면 unscaled score가 커질 수 있다
↓		
04	Softmax saturation	매우 작은 gradient 영역으로 갈 수 있다
↓		
05	Scale	$\sqrt{d_k}$ 로 나눠 magnitude를 완화한다

이 scaling은 Q와 K vector 자체를 unit norm으로 만드는 연산이 아니다

Multi-head는 projection을 따로 학습한다

그림에는 8개 중 두 head만 대표로 표시하며 base model은 $h=8$ 이다



세 attention의 Q, K, V 출처

Cross-attention만 Q와 K,V가 서로 다른 stack에서 온다

Mode	Q source	K,V source	Visible positions
Encoder self	Encoder input	Encoder input	All source
Decoder masked	Decoder input	Decoder input	Current and past
Cross-attention	Decoder state	Encoder output	All source

Cross-attention을 self-attention이라고 부르면 information origin을 잃는다

Causal mask는 score matrix의 위쪽을 막는다

행은 query position, 열은 key position이며 position은 왼쪽에서 오른쪽으로 증가한다

Query / Key	1	2	3	4
1	score	$-\infty$	$-\infty$	$-\infty$
2	score	score	$-\infty$	$-\infty$
3	score	score	score	$-\infty$
4	score	score	score	score

Illegal future score를 softmax 전에 $-\infty$ 로 바꾸면 그 probability는 0이다

Mask와 target shift를 함께 봐야 한다

Training에서도 각 position의 input은 정답 sequence를 한 칸 오른쪽으로 이동한 prefix다

01	Known target	Start token과 이전 target tokens
↓		
02	Shift right	예측할 token은 decoder input에서 제외
↓		
03	Masked self	현재 위치보다 오른쪽 key를 차단
↓		
04	Cross-attention	Source representations는 모두 참조
↓		
05	Next-token loss	해당 위치의 정답 token과 비교

Mask가 current cell을 허용해도 shift가 정답 token의 직접 노출을 막는다

순서를 없앤 것이 아니라 별도로 주입했다

Positional encoding은 token embedding과 같은 d_model 차원이며 element-wise로 더한다

Component	Example role	Combination
Token embedding	단어 identity와 learned features	d_model=512
Position vector	Sin과 cos의 여러 frequency	d_model=512
Model input	Token + position	Addition, not concat
Alternative tested	Learned position embeddings	Nearly identical result

Position vector는 새 token으로 붙는 것이 아니라 embedding에 더해진다

Attention 뒤에는 position-wise FFN이 있다

같은 두-layer network를 각 token position에 독립적으로 적용한다

Stage	Dimension	Operation
Input	$d_{\text{model}}=512$	한 position의 vector
Linear 1	$512 \rightarrow 2048$	$xW_1 + b_1$
Activation	$d_{\text{ff}}=2048$	ReLU
Linear 2	$2048 \rightarrow 512$	$W_2 + b_2$

FFN parameter는 position 간 공유되지만 layer 간에는 서로 다르다

Training과 inference의 경계

Training layer는 positions를 병렬 계산하지만 inference token 생성은 autoregressive하다

01	Training batch	Source와 shifted target sequence 입력
↓		
02	Parallel positions	Mask 아래 모든 target positions 계산
↓		
03	Softmax loss	각 position의 next-token target 비교
↓		
04	Inference	생성한 token을 다음 step input에 추가
↓		
05	Beam search	Paper는 beam 4와 length penalty 0.6 사용

Layer 내부 parallelism이 output token 사이의 sequential generation을 없애지는 않는다

무엇으로 학습했는가

보고된 값은 WMT 2014 translation experiment의 historical setup이다

Task	Sentence pairs	Vocabulary	Batch
EN-DE	about 4.5M	Shared BPE about 37K	about 25K + 25K tokens
EN-FR	36M	Word-piece 32K	about 25K + 25K tokens

Dataset와 tokenization 규모를 빼고 BLEU 숫자만 비교하면 조건을 잃는다

어떤 system과 schedule로 학습했는가

한 machine의 8 NVIDIA P100 GPU에서 base와 big 설정을 따로 보고했다

Setting	Step time	Steps	Elapsed
Base	about 0.4 s	100K	12 h
Big	1.0 s	300K	3.5 days
Adam	beta 1=.9, beta2=.98	warmup 4000	inverse-sqrt decay

시간 수치는 당시 hardware-implementation-model 설정과 함께 해석한다

v7 translation 결과와 내부 불일치

각 숫자에 artifact와 문서 위치를 붙여 서로 다른 statement를 보존한다

Source location	EN-DE BLEU	EN-FR BLEU	Status
arXiv v7 Table 2	28.4	41.8	Transformer big row
arXiv v7 Sec. 6.1 prose	28.4	41.0	Nearby prose
NeurIPS 2017 abstract	27.5	41.1	Separate artifact

41.8과 41.0을 임의로 조정하지 말고 v7 내부의 source-location 차이로 기록한다

Ablation이 말하는 범위

arXiv v7 Table 3의 EN-DE dev result이며 head 수만으로 보편 법칙을 만들 수 없다

Heads	d_k=d_v	BLEU dev	Narrow reading
1	512	24.9	Best listed보다 0.9 낮음
8 base	64	25.8	Base configuration
16	32	25.8	같은 listed best
32	16	25.4	더 많은 head도 하락

이 표는 more heads always help를 증명하지 않는다

Translation 밖의 실험: constituency parsing

4-layer, d_model=1024 Transformer를 두 data regimes에서 평가했다

Training regime	Approx. sentences	WSJ 23 F1	Scope
WSJ only	40K	91.3	Discriminative
Semi-supervised	17M	92.7	Larger corpus

한 추가 task의 결과는 모든 domain으로의 universal generalization guarantee가 아니다

이 논문이 보장하지 않는 것

Architecture claim과 오늘날 모든 Transformer에 대한 claim을 구분한다

오해	Source-grounded boundary
Attention만 존재	FFN·residual·norm·embedding·softmax도 존재
모든 계산이 parallel	Autoregressive token generation은 sequential
$O(1)$ 이 total cost	Full attention cost는 $O(n^2 d)$
Head는 고정 문법 기능	Appendix 그림은 universal guarantee가 아님
현대 model도 동일	2017 encoder-decoder specification의 주장

Attention-only는 recurrent·convolutional sequence layers를 대체했다는 범위의 표현이다

한 token prediction으로 다시 정리

Source context와 target prefix는 서로 다른 attention 경로를 거쳐 다음 token distribution에서 만난다

Stage	What it preserves	What it changes
Encoder self	All source positions	Source context
Decoder masked self	Known target prefix	Target context
Cross-attention	Decoder Q; encoder K,V	Source-target connection
FFN and Add & Norm	Per-position pathway	Feature transform
Linear and softmax	Decoder final state	Next-token probabilities

기여는 attention 중심 encoder-decoder이며 quadratic cost는 남는다